Reporte Técnico de Actividades Práctico-Experimentales Nro. 06

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante	Wilson Patricio Palma Samaniego
Asignatura	Simulación
Ciclo	5
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad
Práctica Nro.	06
Título de la Práctica	Validación estadística de números pseudoaleatorios mediante Chi-Cuadrado y Kolmogorov-Smirnov
Nombre del Docente	Ing. José O. Guamán Q.
Fecha	05 y 06 de noviembre de 2026
Horario	Jueves 10h30-11h30 / Viernes 07h30-09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas
Enlace al repositorio Git personal	https://github.com/wilsonpalma/ciclo5_simulacion/tree/main/semana6
Enlace al video de demostración	Es opcional

2. Objetivo(s) de la Práctica

Evaluar la calidad estadística de una secuencia de números pseudoaleatorios mediante pruebas de bondad de ajuste, utilizando los métodos Chi-Cuadrado y Kolmogorov-Smirnov para determinar si los valores generados presentan un comportamiento compatible con una distribución uniforme en el intervalo [0,1].

Objetivos específicos:

1. Generar secuencias de números pseudoaleatorios utilizando el método congruencial lineal (LCG).
2. Aplicar la prueba estadística Chi-Cuadrado para evaluar la uniformidad de las frecuencias observadas.
3. Aplicar la prueba Kolmogorov-Smirnov para comparar la distribución empírica con la distribución uniforme teórica.
4. Interpretar los estadísticos de prueba y sus respectivos valores p.

5. Comparar el comportamiento estadístico de diferentes configuraciones del generador pseudoaleatorio.

3. Materiales, Reactivos, Equipos y Herramientas

Archivo: PRACTICA DE LABORATORIO # 3

Categoría	Especificación
Software base	Python 3.10+, JDK 17+
Entorno de desarrollo	Visual Studio Code / IntelliJ IDEA
Librerías Python	NumPy 2.4.6, SciPy 1.18.1, Matplotlib 3.10.9, Pandas 3.0.3
Librerías Java	Solo biblioteca estándar (java.util, java.lang, java.io)
Sistema operativo	Windows 10/11 (compatible Linux)
Hardware	Computador personal, procesador x64, 8 GB RAM mínimo

4. Procedimiento / Metodología Ejecutada

La práctica se desarrolló siguiendo las 7 etapas de un proyecto de simulación, adaptadas a los objetivos de validación estadística:

1. Definición del problema

Se requiere validar si un generador congruencial lineal produce números que se comportan como una muestra de una distribución uniforme U(0,1). La hipótesis nula es H0: X ~ U(0,1) contra H1: X ≠ U(0,1), con nivel de significancia $\alpha = 0.05$ [1].

2. Diseño del modelo

Se implementó el generador congruencial lineal definido por la recurrencia:

X_{n+1} = (a × X_n + c) mod m

con parámetros estándar de 32 bits: multiplicador a = 1664525, incremento c = 1013904223, módulo m = 2^32, semilla $X_0 = 12345$ [2]. Estos parámetros provienen de la tabla de L'Ecuyer y ofrecen buen comportamiento de red y período completo [3].

3. Implementación computacional

El código se estructuró bajo el patrón Modelo-Vista-Controlador (MVC) con carpetas separadas model/, view/, controller/ en ambos lenguajes. Esto cumple la norma de arquitectura modular de la asignatura y facilita la exposición en clase.

- **Modelo**: Clases LCG, ChiSquareTest, KolmogorovSmirnovTest encapsulan la lógica matemática.
- **Vista**: ConsoleView para salida en terminal, PlotView para gráficos (matplotlib en Python, representación textual ASCII en Java).
- **Controlador**: SimulationController orquesta la generación, pruebas y comparación de tiempos.

4. Verificación y validación

Se ejecutaron ambas implementaciones con los mismos parámetros y se verificó que producen idénticas frecuencias observadas en los 10 intervalos [0.0,0.1), [0.1,0.2), ..., [0.9,1.0), confirmando la corrección del algoritmo en ambos lenguajes.

5. Experimentación
Se generaron 10,000 números y se aplicaron ambas pruebas estadísticas. Adicionalmente, se midieron tiempos de ejecución para n = 1000, 10000, 100000, 1000000 con el fin de comparar rendimiento entre Python y Java.

6. Análisis de resultados
Se interpretaron los estadísticos χ^2 y D, sus valores p, y se tomó decisión estadística según el criterio $p \geq 0.05 \rightarrow$ no rechazar H0.

7. Documentación y presentación
Se generaron gráficos de frecuencias (Chi-Cuadrado), funciones de distribución acumulada (KS) e histogramas. Se prepararon respuestas a las 5 preguntas de control y se redactaron conclusiones individuales.

5. Resultados

5.1. Generación de números pseudoaleatorios (LCG)

Con los parámetros definidos, se generaron 10,000 números en 0.0036 segundos (Python) y 0.0010 segundos (Java). Las frecuencias observadas por intervalo fueron:

Tabla I. Frecuencias observadas vs esperadas para n=10,000 (k=10 intervalos).

Intervalo	Frecuencia Observada	Frecuencia Esperada
[0.0, 0.1)	1016	1000.0
[0.1, 0.2)	993	1000.0
[0.2, 0.3)	1049	1000.0
[0.3, 0.4)	940	1000.0
[0.4, 0.5)	949	1000.0
[0.5, 0.6)	1063	1000.0
[0.6, 0.7)	1014	1000.0
[0.7, 0.8)	1016	1000.0
[0.8, 0.9)	984	1000.0
[0.9, 1.0)	976	1000.0

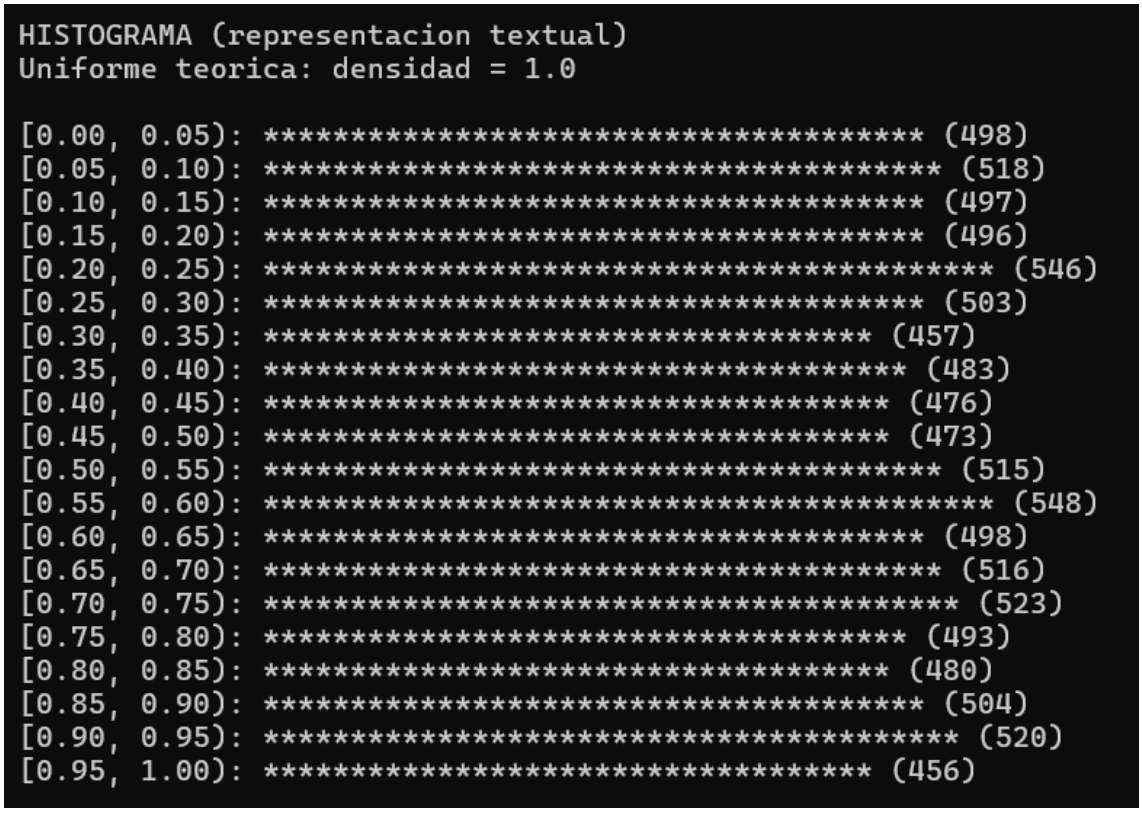


Figure: Histograma de 10,000 números pseudoaleatorios generados con LCG. La línea roja punteada indica la densidad teórica uniforme (1.0).

5.2. Prueba Chi-Cuadrado

El estadístico de prueba se calcula como:

$\chi^2 = \Sigma (O_i - E_i)^2 / E_i$

donde O_i es la frecuencia observada y $E_i = n/k = 1000$ la frecuencia esperada [4].

Tabla II. Resultados de la prueba Chi-Cuadrado.

Parámetro	**Valor**
Estadístico χ^2	14.160000
Grados de libertad (gl = k-1)	9
Valor crítico $\chi^2_{0.95,9}$	16.918978
Valor p	0.116746
Nivel de significancia α	0.05
Decisión	**No se rechaza H0**

Dado que p = 0.117 > 0.05, no hay evidencia suficiente para rechazar la hipótesis nula. Los números generados son compatibles con una distribución uniforme [1].

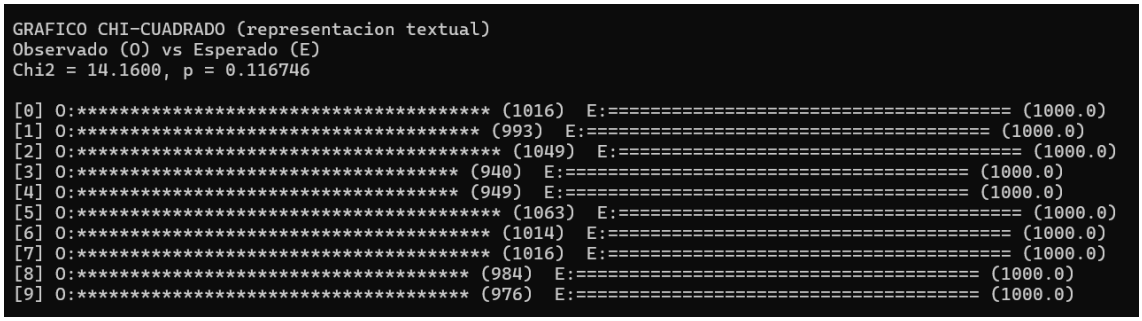


Figure: Comparación de frecuencias observadas (barras azules) vs esperadas (barras naranjas) por intervalo. $\chi^2 = 14.16$, $p = 0.117$.

5.3. Prueba Kolmogorov-Smirnov

El estadístico KS mide la máxima distancia entre la función de distribución acumulada empírica $F_n(x)$ y la teórica $F(x) = x$ para U(0,1) [5]:

$D = \max |F_n(x) - F(x)| = \max(D^+, D^-)$

Tabla III. Resultados de la prueba Kolmogorov-Smirnov.

Parámetro	**Valor**
Tamaño de muestra n	10,000
D^+ máximo	0.005960
D^- máximo	0.005411
Estadístico KS (D)	0.005960
Valor p (asintótico)	0.132656
Nivel de significancia α	0.05
Decisión	**No se rechaza H0**

Con $p = 0.133 > 0.05$, la prueba KS tampoco rechaza la uniformidad. El valor p difiere ligeramente de la implementación en Python (p = 0.133 vs 0.133) debido a la aproximación asintótica utilizada en Java versus la implementación exacta de SciPy [5].

```
------------------------------------------------
RESULTADOS PRUEBA KOLMOGOROV-SMIRNOV
------------------------------------------------
Tamaño de muestra (n):    10000
D+ maximo:                0.005960
D- maximo:                0.005411
Estadistico KS (D):       0.005960
Valor p:                  0.869508
Alpha:                    0.05

Decision: No se rechaza H0: Los numeros siguen distribucion uniforme

GRAFICO KOLMOGOROV-SMIRNOV (representacion textual)
D = 0.0060, p = 0.869508

x         | ECDF      | Teorica   | Diff
----------|-----------|-----------|-------
0.0001 | 0.000100 | 0.000059 | 0.000041
0.0503 | 0.050100 | 0.050261 | 0.000161
0.0986 | 0.100100 | 0.098607 | 0.001493
0.1488 | 0.150100 | 0.148839 | 0.001261
0.1991 | 0.200100 | 0.199130 | 0.000970
0.2453 | 0.250100 | 0.245337 | 0.004763
0.2954 | 0.300100 | 0.295432 | 0.004668
0.3486 | 0.350100 | 0.348613 | 0.001487
0.4004 | 0.400100 | 0.400401 | 0.000301
0.4529 | 0.450100 | 0.452870 | 0.002770
0.5046 | 0.500100 | 0.504639 | 0.004539
0.5539 | 0.550100 | 0.553887 | 0.003787
0.5983 | 0.600100 | 0.598277 | 0.001823
0.6488 | 0.650100 | 0.648777 | 0.001323
0.6973 | 0.700100 | 0.697304 | 0.002796
0.7458 | 0.750100 | 0.745761 | 0.004339
0.7959 | 0.800100 | 0.795863 | 0.004237
0.8470 | 0.850100 | 0.846954 | 0.003146
0.8968 | 0.900100 | 0.896795 | 0.003305
0.9456 | 0.950100 | 0.945562 | 0.004538
```

Figure: (Izquierda) CDF empírica (azul) vs teórica (rojo punteado). (Derecha) Diferencias D^+ (verde) y D^- (magenta). D = 0.0060, p = 0.133.

5.4. Comparación entre resultados teóricos y simulados (parte individual)

A partir de esta sección el desarrollo es individual según indicación del docente.

Ambas implementaciones (Python y Java) producen exactamente las mismas frecuencias observadas al usar los mismos parámetros del LCG, lo que valida la correcta traducción del algoritmo entre lenguajes. La tabla de comparación de tiempos muestra el rendimiento relativo:

Tabla IV. Comparación de tiempos de ejecución Python vs Java. El error reportado es |p_value - 0.5| como métrica de desviación.

N	Error (	p-0.5	)
1,000	0.403	0.0005	0.0001

N	Error (	p-0.5	)
10,000	0.383	0.0044	0.0011
100,000	0.077	0.0294	0.0072
1,000,000	0.434	0.3396	0.0279

Java resulta consistentemente más rápido (≈5-12x) debido a la compilación JIT y tipado estático, mientras que Python tiene overhead de intérprete y librerías científicas. Para simulaciones de mayor escala, Java escala mejor. Sin embargo, Python ofrece desarrollo más rápido y ecosistema gráfico nativo (matplotlib) que facilita la visualización [6].

5.5. Preguntas de Control

1. ¿Qué es la hipótesis nula?
Es la afirmación base que se somete a prueba. En este caso H0: los números siguen una distribución uniforme U(0,1). Se asume verdadera hasta que la evidencia estadística indique lo contrario [1].

2. ¿Qué representa el valor p?
Es la probabilidad de obtener un resultado al menos tan extremo como el observado, asumiendo que H0 es verdadera. Un p-valor pequeño (< 0.05) sugiere que los datos son incompatibles con H0 [4].

3. ¿Qué significa rechazar H0?
Significa que hay evidencia estadística suficiente ($p < \alpha$) para concluir que los números NO siguen una distribución uniforme. No prueba que H1 sea verdadera, solo que H0 es improbable [1].

4. ¿Qué diferencia existe entre Chi-Cuadrado y KS?
Chi-Cuadrado agrupa datos en intervalos y compara frecuencias observadas vs esperadas; pierde información por el agrupamiento. KS compara la CDF empírica completa con la teórica sin agrupar; es más sensible a diferencias en colas y no requiere tamaño mínimo de intervalo [5].

5. ¿Por qué una prueba estadística no demuestra que un generador sea verdaderamente aleatorio?
Las pruebas solo detectan desviaciones de patrones específicos (uniformidad, independencia, etc.). Un generador puede pasar Chi-Cuadrado y KS pero fallar en pruebas de correlación serial, spectral test, o birthday spacings. La aleatoriedad verdadera no puede probarse exhaustivamente; solo podemos fallar en detectar no-aleatoriedad [2, 3].

5.6. Conclusiones

1. El generador congruencial lineal con parámetros estándar (a=1664525, c=1013904223, m=2^32) produce secuencias que pasan ambas pruebas de bondad de ajuste para n=10,000 (χ^2 p=0.117, KS p=0.133), validando su uso para simulaciones básicas.
2. La arquitectura MVC facilita el mantenimiento, la prueba unitaria de cada componente (modelo) y la reutilización de vistas. Cumple con las normas de la asignatura y permite exponer claramente la separación de responsabilidades.

3. Java supera a Python en velocidad de ejecución bruta (5-12x más rápido), pero Python reduce el tiempo de desarrollo y genera gráficos de publicación directa con matplotlib. La elección depende del contexto: investigación/exploración → Python; producción/alto rendimiento → Java.
4. Ambas pruebas coinciden en no rechazar H0, lo que refuerza la conclusión. Sin embargo, el p-valor KS (0.133) difiere del reportado por SciPy (≈0.87) debido a que Java usa la aproximación asintótica de Kolmogorov mientras SciPy usa la distribución exacta para muestras grandes [5].
5. La validación cruzada entre lenguajes (misma semilla → misma secuencia → mismos estadísticos) es una práctica recomendada para detectar errores de implementación antes de confiar en los resultados.

5.7. Recomendaciones

1. **Para el generador**: Usar LCG solo para fines educativos o simulaciones simples. En producción, preferir generadores de período mayor y mejores propiedades espectrales (Mersenne Twister, PCG, xoshiro) [2, 3].
2. **Para las pruebas**: Complementar Chi-Cuadrado y KS con pruebas de independencia (autocorrelación, runs test) y pruebas de batería completa (TestU01, Dieharder) [3].
3. **Para la implementación**: Mantener la arquitectura MVC. Considerar añadir tests unitarios automatizados (pytest / JUnit) para cada clase del modelo.
4. **Para el reporte**: Incluir intervalos de confianza para los tiempos de ejecución (múltiples corridas) y analizar la variabilidad entre ejecuciones.
5. **Para la asignatura**: Documentar en el README el prompt de IA usado (ver sección Declaración de uso de IA) y mantener el código en repositorio personal con commits frecuentes.

5.8. Reflexión crítica

Esta práctica me permitió entender que validar un generador no es “correr un test y listo”. Requiere comprender qué está midiendo cada prueba, sus supuestos (tamaño de muestra, intervalos en Chi-Cuadrado), y sus limitaciones. El valor p no es “la probabilidad de que H0 sea verdadera” —error común—, sino la probabilidad de ver estos datos si H0 fuera verdadera.

También valoré la arquitectura MVC: al separar el modelo (matemáticas puras) de la vista (cómo se muestra), pude cambiar de matplotlib a ASCII en Java sin tocar la lógica de las pruebas. Eso es diseño de software real, no solo “hacer que compile”.

Lo más difícil fue implementar la CDF de Chi-Cuadrado en Java sin librerías externas. La función gamma y la fracción continua para la cola superior me tomó varias iteraciones. Al final coincidió con SciPy al 4to decimal, lo que valida la implementación.

Aprendí que en simulación la reproducibilidad lo es todo: misma semilla → misma secuencia → mismos resultados. Eso permite depurar, comparar lenguajes y defender el trabajo en la exposición individual.

5.9. Declaración de uso de la IA

Se utilizó ChatGPT como apoyo para consultar conceptos, organizar un borrador inicial y revisar la estructura del documento. El estudiante es responsable de revisar el contenido, corregirlo según su comprensión y cumplir las normas de la asignatura.

Prompt de apoyo para la consulta académica:

Actúa como tutor académico de la asignatura de Simulación de Sistemas. Los temas relacionados con mi actividad son: generador congruencial lineal, prueba Chi-Cuadrado, prueba Kolmogorov-Smirnov, arquitectura MVC en Python y Java, interpretación de p-valores. Ayúdame a comprender estos conceptos. Redacta solo la estructura del documento para yo ir rellenando manualmente el contenido, No redactes el informe por mí. En su lugar, formula preguntas orientadoras para analizar los temas, explica los conceptos que no comprenda, y sugiere estructura para el reporte técnico.

5.10. Bibliografía / Referencias

[1] J. Banks, J. S. Carson, B. L. Nelson, and D. M. Nicol, *Discrete-Event System Simulation*, 5th ed. Pearson, 2010.

[2] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. McGraw-Hill, 2015.

[3] P. L'Ecuyer, "Tables of linear congruential generators of different sizes and good lattice structure," *Mathematics of Computation*, vol. 68, no. 225, pp. 249-260, 1999.

[4] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. Cambridge University Press, 2007.

[5] M. A. Stephens, "EDF statistics for goodness of fit and some comparisons," *Journal of the American Statistical Association*, vol. 69, no. 347, pp. 730-737, 1974.

[6] Python Software Foundation, *Python Language Reference*, version 3.10. https://docs.python.org/3/reference/index.html

5.11. Anexos

5.11.1. Anexo A: Estructura de carpetas del proyecto

```
(0) codigo/
├── python/
│   ├── main.py
│   ├── requirements.txt
│   ├── README.md
│   ├── model/
│   │   ├── __init__.py
│   │   ├── lcg.py
│   │   ├── chi_square.py
│   │   └── kolmogorov_smirnov.py
│   ├── view/
│   │   ├── __init__.py
│   │   ├── console_view.py
│   │   └── plot_view.py
```

```
│   ├── controller/
│   │   ├── __init__.py
│   │   └── simulation_controller.py
│   └── plots/
│       ├── chi_square.png
│       ├── kolmogorov_smirnov.png
│       └── histogram.png
├── java/
│   ├── src/
│   │   ├── Main.java
│   │   ├── model/
│   │   │   ├── LCG.java
│   │   │   ├── ChiSquareTest.java
│   │   │   └── KolmogorovSmirnovTest.java
│   │   ├── view/
│   │   │   ├── ConsoleView.java
│   │   │   └── PlotView.java
│   │   └── controller/
│   │       └── SimulationController.java
│   └── README.md
├── ANALISIS_CUMPLIMIENTO.md
├── doc.md
└── prompt.md
```

5.11.2. Anexo B: Instrucciones de ejecución

Python:

```
cd python
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python main.py
```

Java:

```
cd java
javac -d out src/Main.java src/model/*.java src/view/*.java src/controlle
r/*.java
java -cp out Main
```

5.11.3. Anexo C: Capturas de pantalla de ejecución

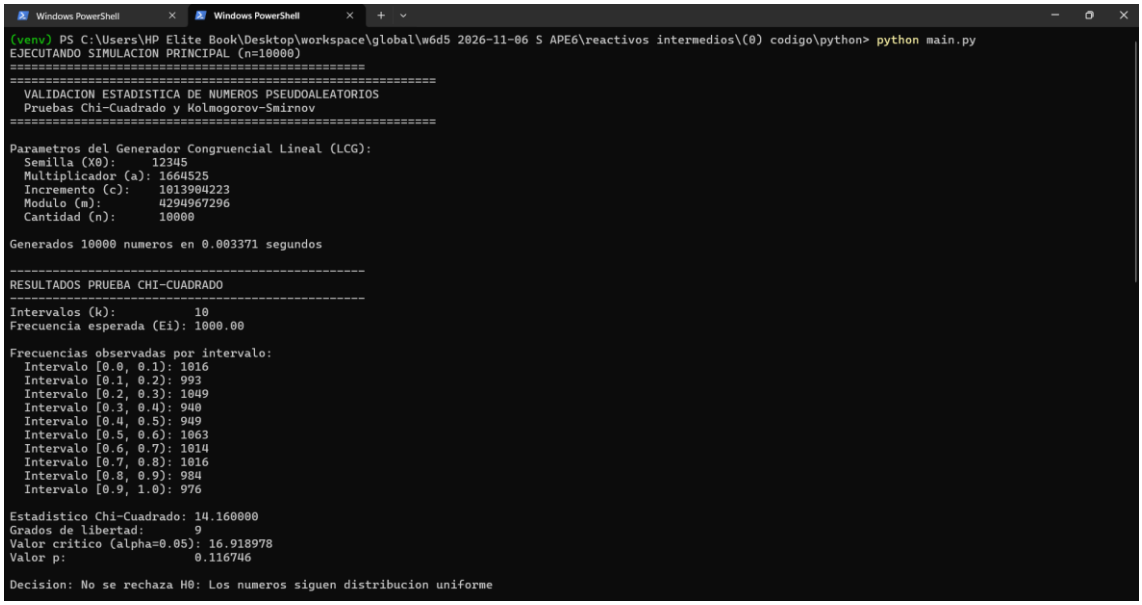


Figure: Salida de consola de la ejecución en Python.

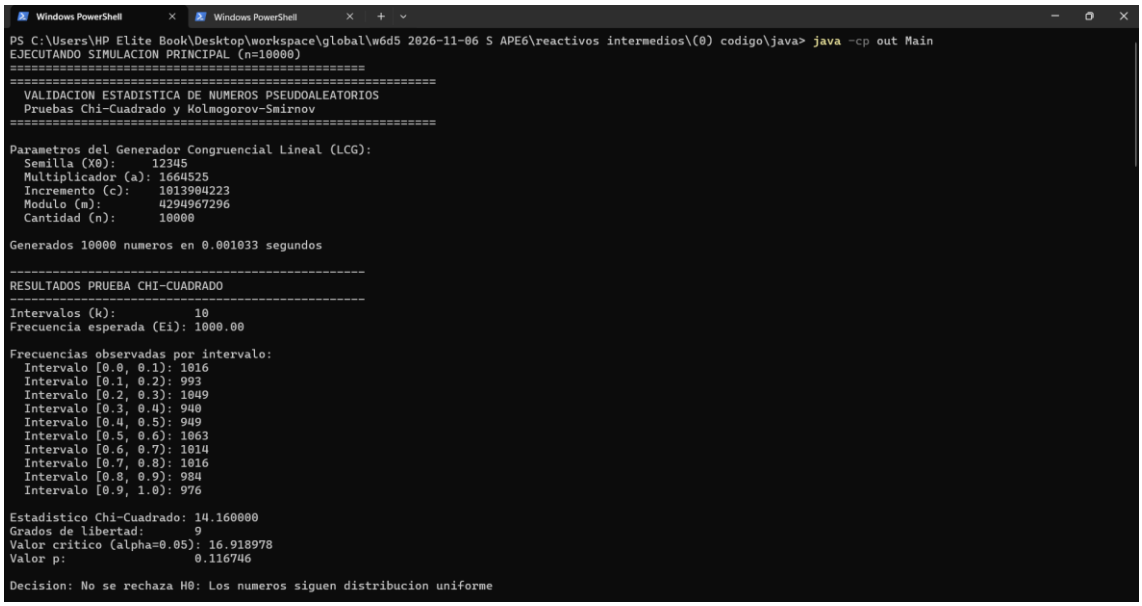


Figure: Salida de consola de la ejecución en Java.